Smart placement of a two-arm assembly for an everyday object manipulation humanoid robot based on capability maps

Israel Chaves-Arbaiza, Daniel García-Vaglio, Federico Ruiz-Ugalde
ARCOS-Lab. Autonomous Robots and Congnitive Systems Laboratory
Escuela de Ingeniería Eléctrica. Instituto de Investigaciones en Ingenería. Universidad de Costa Rica
israel.chaves@ucr.ac.cr, daniel.garciavaglio@ucr.ac.cr, federico.ruizugalde@ucr.ac.cr

***Abstract*—A robot that will execute everyday object manipulation tasks needs a competent body that can handle as many different objects as possible in as many ways as possible. To accomplish this, we must design the robot body in such a way that will allow it to achieve many different ways to manipulate the objects. In this work, we present a design method to mount a two-arm assembly in a torso and a mobile base (to complete a humanoid robot) taking into account a voxelized structure of points to reach and a variable orientation and relative position of both arms. We make use of the concept of capability maps to calculate a score for different possible two-arm assembly placements. An object to manipulate is selected. In particular, we are using the object size as the most important manipulation feature. We test many possible arm mounting configurations with many different object positions and we run an object reaching simulation for each of these combinations. The successfulness of each of these trails is used to derive a capability map. A score for each map (that corresponds to a different arm mounting configuration) is derived and used to select the final placement of the robot. With this work, we present an effective process to determine the best arms bases placement accord to the collaborative reaching results for a two-arm humanoid robot.**

I. Introduction

If a humanoid robot is going to work as an assistant for humans in every day tasks it is necessary for the robot to have a dexterous manipulation system, that allows it to interact safely with humans and other robots. There are many tasks that require the robot to be able to perform bi-manual objects manipulations, like opening a jar, or moving a big object (like a pot) from one place to another. Therefore, it is important for the robot to have a body that is able to execute two handed manipulation tasks as best as possible.

Many humanoid robots are designed to be as human-like as possible but having a anthropomorphic configuration does not necessarily imply that it will be able to have the best bi-manual manipulation capabilities. For a robotic system with two arms, with or without a mobile system (legs or wheels), it is vital to ensure the robot to manipulate objects using both arms efficiently, taking advantage of maximum space and poses that every arm can reach. Unfortunately, it is common to assemble a humanoid robot only trying to copy a human as much as possible and leaving completely the fact that the robot body is an artificial machine that may have differences, with respect to a human body, that may degrade its manipulation capabilities if its assembly configuration is just a copy of a human. Some other assembly configurations may produce better manipulation capabilities.

In this paper we offer a method for helping design a humanoid robot arm-torso assembly , where our main design criteria is to maximize the two-arm shared reachability map, which is computed by generating a mixed score based on the calculations of a typical one arm capability map. Also, in our approach, not only is the inverse kinematics of the final goal taken into account for the calculation of two-arm shared reachability map, but also the whole reaching movement of the arms from an initial away position. To be precise, the same system that controls our reaching and manipulation actions on our previous real robot is used (in simulation) to calculate our reachability map. In this way, we are taking into account a real-case use of the arms for deriving our reachability maps and not the limited position inverse kinematics that is commonly used.

It is necessary to clarify that we are considering just the size of the object, not the shape, because there are multiple grasping tools that we are evaluating to use in our robot. So, this study is centered in tasks of reaching objects.

In order to simplify the problem of manipulability measurements, goals are approximated to poses to reach in half a sphere. The simulation process consists of multiple arm base placements for both arms in a symmetric configuration and each arm trying to reach a certain number of goals poses (left arm to left semi-sphere and right arm to right semi-sphere). Once the simulation generates the results of all goal poses that were successfully or not successfully reached by each arm, it creates the score for each reachability map and also a global score, taking in consideration every reachability index for correspondent left and right semi-spheres to get a global index for that sphere of goal poses. This way, it is possible to choose a better base arm placement and configuration according to the areas with higher capabilities for dual manipulation tasks.

II. Related Work

In [3] presents the design considerations for the upper part of DLR Justin. Which has a very similar manipulator system to our robot. Some of the requirements were to have a system that could be able to reach an object on the floor, on a shelf, and capability for two-hand grasping. In this robot, the arms are mounted on the torso on a 60 degrees tilted configuration,

Authorized licensed use limited to: Universidad de Costa Rica (UCR). Downloaded on June 18,2021 at 16:16:41 UTC from IEEE Xplore. Restrictions apply.

which allows for an operation far from the arm limits and singular configurations.

In [6] the capability map is used to determine a manipulator configuration in order to perform constrained trajectories with the end effector. First the capability map is computed, and using pattern recognition techniques areas within the workspace are found where this required trajectories can be performed. The result is a set of possible trajectories in these areas. Then manipulator placements are computed and the corresponding trajectories are checked for collisions.

To have a good two arm manipulation performance, one wants to maximize the share workspace area between the arms. This means that a good arm base placement is required (shoulders). Robot base positioning for manipulation tasks is studied here [5], [2], [4] and [1].

In [4] an inverse capability map is computed, in order to find a robot base pose that can help to grasp an object. Given a grasping pose the possible robot base positions are computed (discrete solutions), and for every position a set of possible orientations. So a map of possible base 2D poses is computed for the given grasp pose. Because orientation information is stored in every map point, this structure is called Oriented Reachability Inverse Map (Oriented Reachability Map, and Inverse Reachability Map).

An orientation based capability map is proposed in [2]. The problem with conventional capability maps is that they depend on the end effector that was used to compute them. The orientation-based capability map allows online end effector frame extension. experimental results show that orientation-based capability map is as fast as the conventional (position based) capability map. This approach is very useful when the robot is required to use different tools, and the end effector pose has to be computed for each tool. It only depends on the robot arm structure, and not on the tool that is used.

This approaches try to find the robot base position in a 2D plane, but the shoulder is static respect to its base.

In [5] an approach to improve inverse kinematics is presented. They extend the capability map, by adding IK solutions, this new map is called an IK-Map. Reference IK solutions are stored in every point of the capability map, so that this information can be used to seed iterative (gradient descent) IK solvers during online processing. In this paper a 10 DoF (degrees of freedom) robot is used, 2 hip joints and 8 arm joints. Because they are taking into account the 2 hip joints, they are able to change shoulder pose, in order to improve workspace area.

This solution is very ineffective in terms of memory consumption, because an IK solution has to be store in every point of the capability map. Offline processing is also very slow, according to the paper for generating a IK-map from 100-million samples (10 DoF), took around 9 hours on a standard Linux PC utilizing 8 cores. While on online processing, the end effector pose is searched in the IK-map, and the reference IK solution is used as seed to an iterative IK solver. This approach has been proven to help speed up IK online processing.

In [1] an approach to select the optimal stance location using the Inverse Reachability Map for a whole humanoid robot is presented. A manipulability measure is used for every point with 1. Where J is the Jacobean matrix of the respective kinematic chain in configuration q σ_i is the i-th singular value.

$$\omega = \left(det[J(q)J^T(q)]\right)^{\frac{1}{2}} = \sigma_1\sigma_2...\sigma_n \tag{1}$$

A whole body capability map is computed, taking into account arm and legs joints. Then, the inverse of this capability map is computed, in order to obtain possible feet 2D poses for a given grasp pose. This was a good approach for the platform that was used (Nao), that had only 5 DoF on each arm, so an intelligent stance selection is necessary to increase manipulability.

All the above approaches only take into consideration one arm. But we are working with a 2-arm reachability map, where wants to improve workspace area and prevent collisions between the arms.

In [8] path planning for two arms is studied. The capability maps of both arms are computed and then, they are separated on the disjoint and shared subsets. Manipulations tasks are simulated and then executed on a real robot. Tasks can be computed for parallel or sequential execution, but when working on the shared workspace, tasks are executed only in a sequential way.

III. Hardware and Software Infrastructure

Our humanoid robot proposed design is shown in figure 1. The robot is composed of a omnidirectional mecanum-wheel mobile platform, a torso, two kuka LWR 4+ arms, two Wessling Robotics hands and a perceptive head. The head is in turn composed of a neck, a perceptive skull and an emotional face.

The torso is composed of a space for storing computers and controllers for the robot and also of a motorized lifting structure that allows the robot to move the arms up and down. On top of that structure the arms will be mounted in a particular configuration (Refer to figures 2 and 3).

Each arm can be mounted at a certain “y” distance from the center of the robot, such as that each arm gets separated from the other. We call this the “y offset”. We decided that this offset should be the same for both arms to maintain symmetry (a proper look for humans to accept the robot). If this would not be symmetrical, the simulation would take much longer.

Also each arm can be mounted in a “pitch” (tilting up and down) and “yaw” (moving outwards of inwards) configuration. We also decided, for the same reason as above, that these parameters should be symmetrical. Pitch will go between looking forward to looking up (0 to 90 degrees). Yaw will go between looking forward to looking to the side (0 to 90 degrees).

The last parameter to explore is the roll of each arm. Each arm can be mounted rotated in its own axis. This is important because the first joint of the arm can not rotate indefinitely, and it will reach a joint limit depending on this mounting configuration. Because both arms are not symmetrical (both are “right” arms), then each mounting roll angle must be explored separated: “roll-right”, “roll-left”.

Authorized licensed use limited to: Universidad de Costa Rica (UCR). Downloaded on June 18,2021 at 16:16:41 UTC from IEEE Xplore. Restrictions apply.

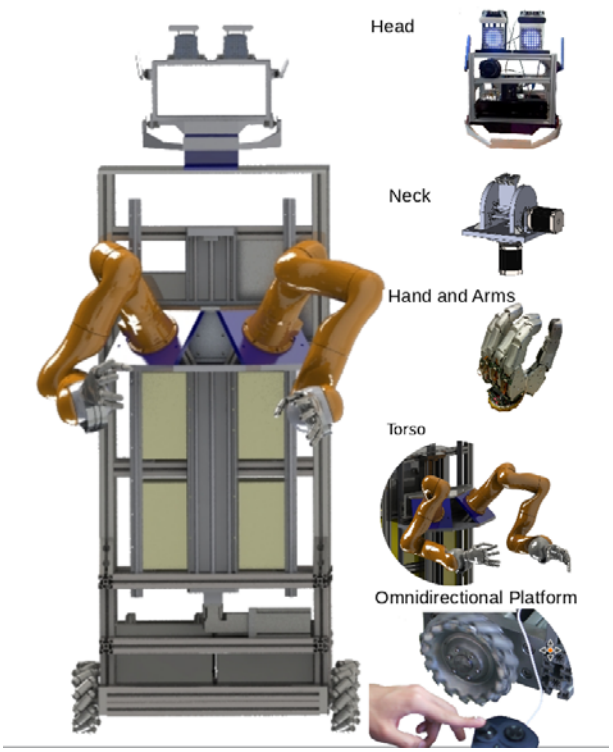


Fig. 1: ARCOSBOT humanoid robot.

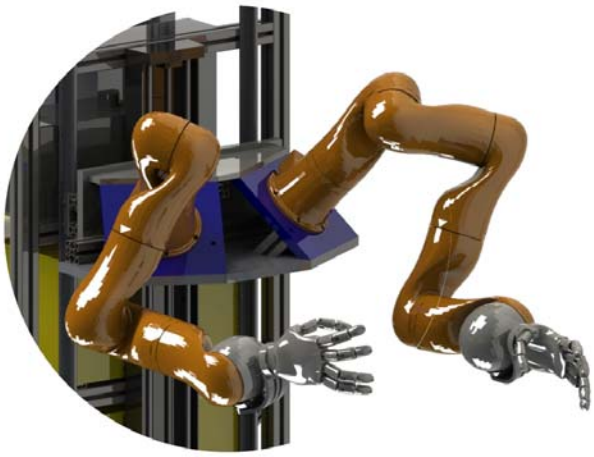


Fig. 2: ARCOSBOT shoulder.

Another important consideration is about the hand, we are setting the both hands.

The software infrastructure that our robot uses is a vector field velocity-based closed-loop weighted-damped least-squares inverse kinematics (vfclik) (see Fig. 4). It manages each arm as a separate system. We are currently incorporating to our robot the torso up and down movement. This torso movement is shared between both arms. For simplicity we decided to assign usage and control of this joint to the right arm. The left arm uses this joint to read its current position but can not control it. This system can accept joint or cartesian control commands for each arm individually. It can also take a tool frame that it adds to the end-effector of each arm, and

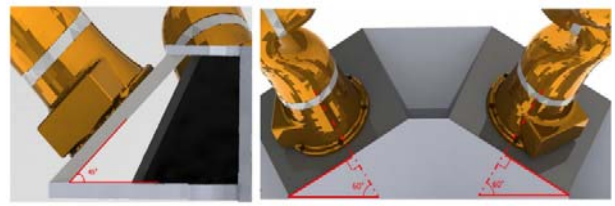


Fig. 3: ARCOSBOT shoulder configuration angles.

then it uses this new point to control arm movement. The system also uses the weighted-damped least-squares inverse kinematics to use in much less intensity the torso joint.

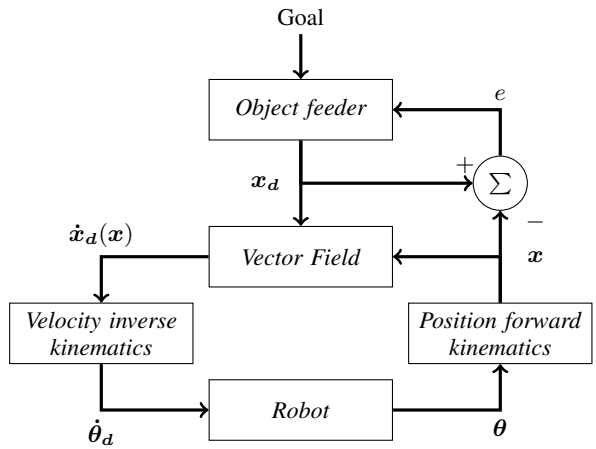


Fig. 4: Partial block diagram of the VFCLIK system. $\boldsymbol{x}_d$, $\dot{\boldsymbol{x}}_d(\boldsymbol{x})$ and $\boldsymbol{x}$ represent the desired intermediate point, the velocity vector and the current position in work space. $\dot{\boldsymbol{\theta}}_d$ and $\boldsymbol{\theta}$ correspond to the velocity and angular vectors in joint space. e is the error used to determine how near to the current desired point is the end-effector.

This system was modified to be able to run many robot instances to parallelize goal trials and to execute the goal commands as fast as possible to be able to run hundreds of thousands of goals.

IV. Goal and Simulation Generation

In order to find out which arm mounting configuration is the best we run a search: in each "y-offset", "pitch", "yaw", "roll-right" and "roll-left" configuration we run a grid of goals changing its x and y position (x: to the front of the robot, y: to the sides) and also changing its roll and yaw goal orientations.

The process is as follows: we run a goal generation script that generates a long list of goals for each arm mounting configuration. This list of goals is then fed to a system that manages and starts multiple instances of the robot running at high speed. This system dispatches a goal to each instances that finished a previous goal. With our mini-cluster we were able to achieve 10 goals per second.

We use the following goal generation configuration for the mounting base parameters:

- y-offset range: 0.1 to 0.25, steps: 0.07

Authorized licensed use limited to: Universidad de Costa Rica (UCR). Downloaded on June 18,2021 at 16:16:41 UTC from IEEE Xplore. Restrictions apply.

- pitch range: 0 to 90 degrees, steps: 45 degrees
- yaw range: 90 to 0 degrees, steps: 45 degrees
- roll range: -180 to 180 degrees, steps: 90 degrees

And the following goal sampling:

- x range: 0.3m to 0.8m, steps: 4.9cm
- y range: -0.25m to 0.25m, steps: 4.5cm
- roll range: -90 to 90 degrees, steps: 45 degrees
- yaw range: -90 to 90 degrees, steps: 45 degrees

In total, the system generated 405504 goals that took around 11 simulation hours.

The goals for each arm are totally opposed through a "sphere". The idea is that the robot will usually try to grasp the object, and this is probably the best strategy for avoiding self collisions. Another important self imposed limitation is that all the goals are at a height of 1.2 meters. We assume the robot is usually working around a common table height. Also, since the robot has a torso joint, object height shouldn't be that important because the robot can adjust the torso height if necessary.

An important consideration of this paper is the fact that the simulation realizes the complete reach process for each pose with both arms, instead of the solution for inverse kinematics equation for each pose. So, in order to increase simulation speed, one can take advantage from reducing arm's inertia to make the robot move faster.

When manipulating an object, the first step is to visualize it and make an estimation of its size, form and pose in step. For this, the robot is required to take its arms outside the visual space, as shown in figure 8 . This initial arm pose is used as the arbitrary far point from which arms start their trajectory to the goal on the sphere surface. Figure 8 shows the robot trying to reach opposite points of the sphere.

Figure 8 shows the set of goals (vector poses to reach a point of the semi-sphere's surface) that the robot must reach. They are represented as a frame. Goals are computed with a $\pi/6$ step in XZ angles. The task, of positioning the arms in the start position and then trying to reach a determined goal on the sphere surface, is repeated for every goal pose.

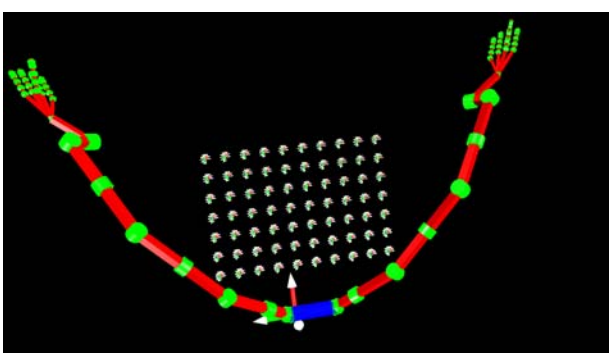


Fig. 5: Goals grid from above.

When each arm tries to reach every pose, it records the result in a new variable (one for every arm) besides the 7 variables at the input. Once the simulation is completed, a reachability metric is been used for the given goals based on the fraction of discrete poses in the semi-sphere that are reachable for the corresponding arm. In equation (2), the reachability measure D, where the number of discretized goal poses is represented by N and the number of reached poses is R.

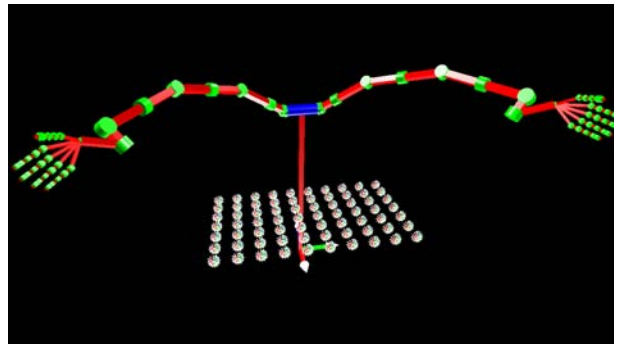


Fig. 6: Goals grid for a particular arm mounting configuration.

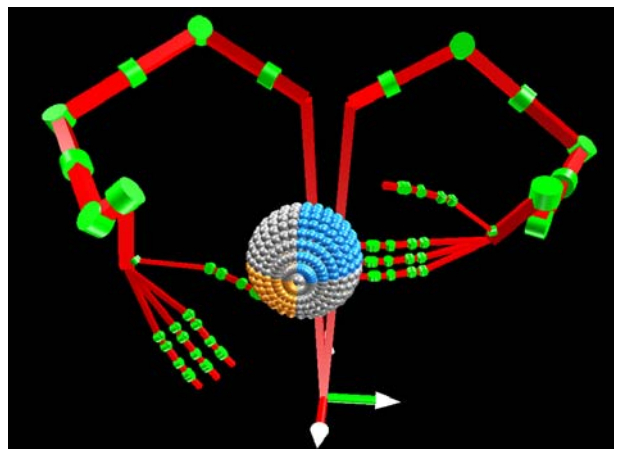


Fig. 7: Sphere division in quadrants. Orange: right arm. Blue: left arm

$$D = (\frac{R}{N}) * 100 \tag{2}$$

With the calculated D, semi-spheres are assigned to different colors, where blue represents higher reachability, the goals with most reachable poses of the semi-sphere surface, and red represents lower reachability. [7] shows the corresponding color, for the reachability index from red to blue, in terms of increasing.

In this paper, we propose a dual capability map, taking into consideration both D indices for the same sphere, creating a map with two colors spheres for each goal. With the dual capability map, its easier to detect better areas where the robot can use both arms to achieve collaborative tasks.

V. Results

We made a simulation with 405504 goals for the robot to reach. They all were performed at a standard height of a table.

Authorized licensed use limited to: Universidad de Costa Rica (UCR). Downloaded on June 18,2021 at 16:16:41 UTC from IEEE Xplore. Restrictions apply.

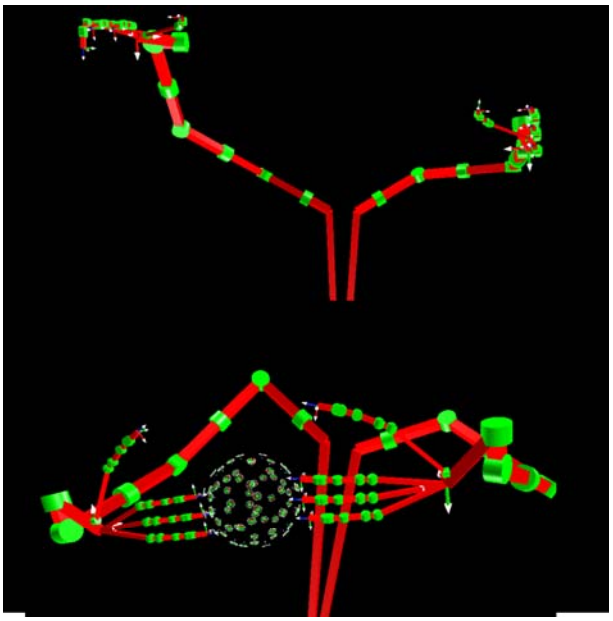


Fig. 8: Above: robot initial pose. Below: robot reaching the sphere

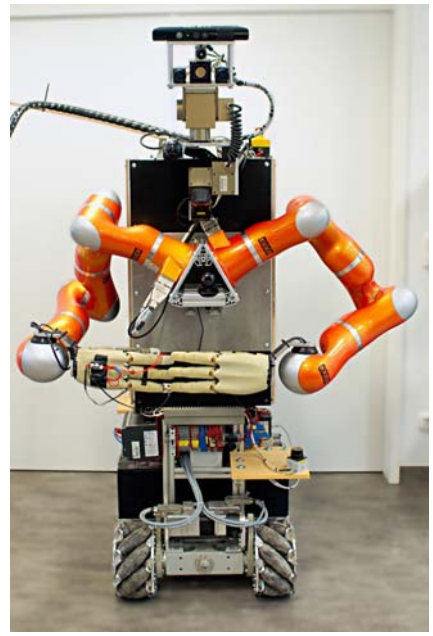


Fig. 9: TUM-Rosie

For each goal, we stored a "succes" or "failed" value, if the left and the right arms were able to reach the goal. It is important to note that we were performing reach actions, and not just solving inverse kinematics. This lets us have the certainty that not only the robot can have the arms at that position, but it can also move to this position correctly.

In other robotic platforms (for example TUM-Rosie), the "pitch" and "y-offset" was varied to have better manipulation capabilities. But "yaw" is normally set to be $90°$. To test different "yaw" values we fixed the rest of the shoulder configuration variables to the ones used in TUM-Rosie and simulated the reach tests explained before. The TUM-Rosie configuration is $60°$ of pitch and the shortest y-offset such that the bases do not collide, as shown in figure 9. In figure 10 we show how the relationship between the score and yaw. The maximum is reached when yaw is approximately $60°$.

The results of the goals on the table are what we call a dual reachability map. We generated one of these maps for every possible shoulder configuration. This map is useful to see the shared reachable-space of both arms over the table (see figure 11). Figure 11a has a very poor dual-hand shared reachability, because there are very few points with a high score, meanwhile on the map of figure 11b there is a very strong dual-hand shared reachability.

For each of these maps, it is possible to calculate the total score, this is the addition of the individual scores of all points. This metric lets us compare between shoulder configurations to find the best one. The result was very similar to what intuition tells, the optimal configuration has a "y-offset" of 0.25m. A pitch of $45°$ radians and a yaw of $90°$ radians. The optimal roll for both sides is the same, and equal to zero. The optimal obtained configuration is shown if figure 12 where the robot has the arms fully extended.

VI. Discussion and Future Work

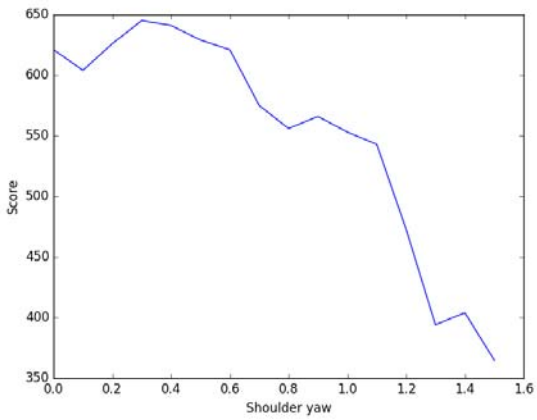
Using a simulation of an execution of many goals and possible shoulder configurations we were able to arrive to a configuration that is optimal with respect to our given goal executions. This result takes into account two-handed manipulation as something more important and more valuable than single handed manipulation. Our system also takes into account the whole reaching movement using a vectorfield closed-loop velocity weighted-damped least-squared inverse kinematics instead of the simpler position inverse kinematics that is usually used for calculating the capability maps. This system also takes into consideration arm joint limits and many more real-world considerations for goal reaching. We consider important that the simulation takes into account as many real world considerations as possible; this is important to properly use this information to direct our robot design.

We hope to improve our results by running a more exhaustive simulation with higher granularity in all the dimensions.

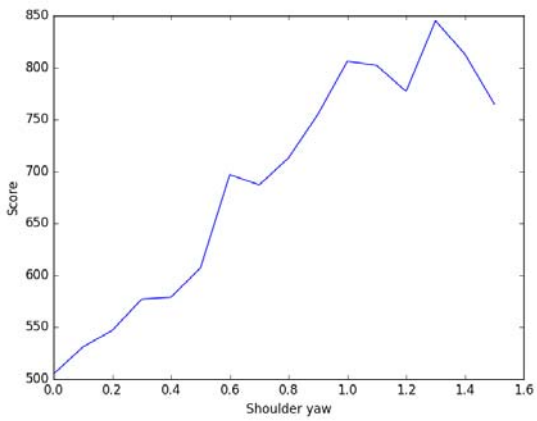
Possible future improvements/uses:

1) Use more granularity for all the simulation dimensions (goals and base configurations) to obtain a more precise arm base mount configuration. We are currently running the simulation system in our mini-cluster to improve the results presented in this paper.
2) The hand kinematics to analyze a dual reachability/-graspability map for sliding and/or grasping tasks.
3) Consider self- and inter-collisions for both arms reaching the goals.
4) Consider selecting the initial starting movement position in cartesian instead of joint space. The initial arm configurations varies considerably as the base configurations change during the simulation. This initial arm setup is important to permit the perception system look at the table while the arms are not in the

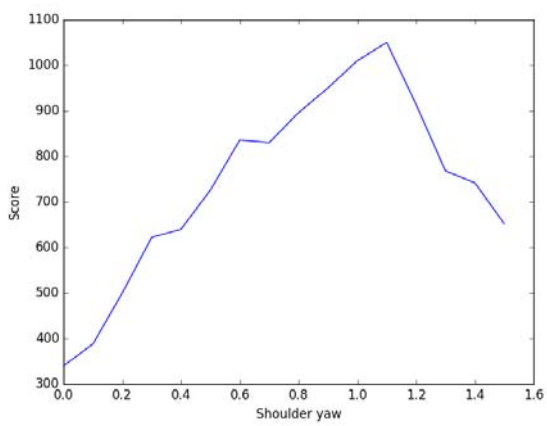
Authorized licensed use limited to: Universidad de Costa Rica (UCR). Downloaded on June 18,2021 at 16:16:41 UTC from IEEE Xplore. Restrictions apply.



(a) Left arm reachability score against shoulder yaw

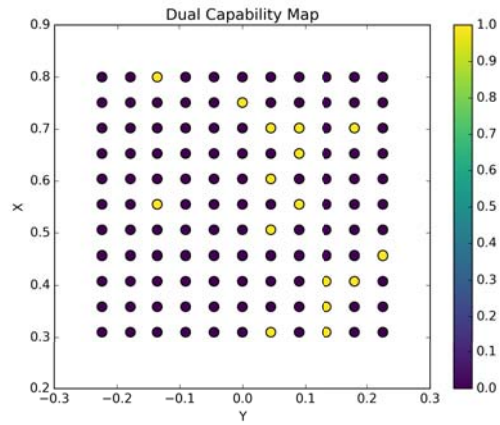


(b) Right arm reachability score against shoulder yaw

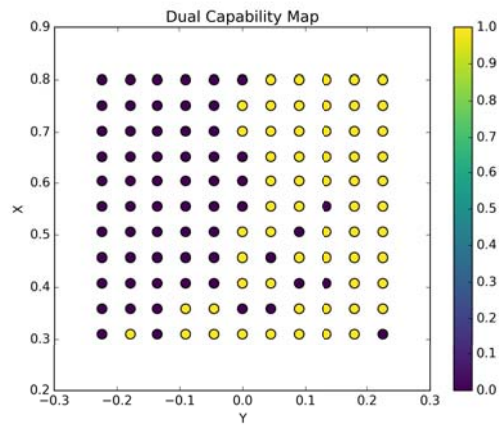


(c) Dual reachability score against shoulder yaw

Fig. 10: Score as a function of shoulder yaw



(a) Map of low integration



(b) Map of high integration

Fig. 11: Comparison of reachability maps

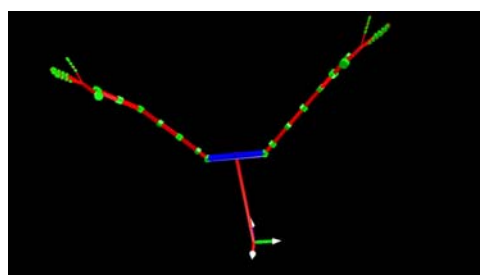


Fig. 12: Optimal shoulder configuration for dual manipulation over a table

way.

5) Once the robot is built with the selected arm base configuration, verify that goal reachability is similar than the one expected in this paper.
6) Consider the shape of the object and the grasping tools to determine the best configuration setup for both arms, depending of the tools (not necessary hands) and the principal manipulation tasks of the robot.

Authorized licensed use limited to: Universidad de Costa Rica (UCR). Downloaded on June 18,2021 at 16:16:41 UTC from IEEE Xplore. Restrictions apply.

7) Do a two-level simulation where in the first pass, for each arm mounting configuration a reachability map is generated and use to find out the best reachable areas and then generate a new coordinate limits and steps to improve goal resolution around this parts of the reachable space for a second simulation pass.
8) Develop a system, that uses the reachability results from this work, to determine which arm would be best to use for a particular manipulation task using goal score level.

VII. Acknowledgements

The present paper is a partial result of a research project entitled ”Torso, mobile platform and head integration for a humanoid robot, 731-B7-220.” funded by the Research Agency of the University of Costa Rica (Vicerrectoría de Investigación). Additional funding and support is provided by the University of Costa Rica Dean’s office, Electrical Engineering department and Engineering Research Institute. We would like to acknowledge all the support given by many members of the ARCOS-Lab. Special thanks to Ariel Mora and Andres Alvarado, for their efforts in many aspects of this paper.

References

[1] F. Burget and M. Bennewitz, “Stance selection for humanoid grasping tasks by inverse reachability maps,” in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, May 2015, pp. 5669–5674.

[2] J. Dong and J. C. Trinkle, “Orientation-based reachability map for robot base placement,” in *Intelligent Robots and Systems (IROS), 2015 IEEE/RSJ International Conference on*, Sept 2015, pp. 1488–1493.

[3] C. Ott, O. Eiberger, W. Friedl, B. Bauml, U. Hillenbrand, C. Borst, A. Albu-Schaffer, B. Brunner, H. Hirschmuller, S. Kielhofer, R. Konietschke, M. Suppa, T. Wimbock, F. Zacharias, and G. Hirzinger, “A humanoid two-arm system for dexterous manipulation,” in *2006 6th IEEE-RAS International Conference on Humanoid Robots*, Dec 2006, pp. 276–283.

[4] N. Vahrenkamp, T. Asfour, and R. Dillmann, “Robot placement based on reachability inversion,” in *Robotics and Automation (ICRA), 2013 IEEE International Conference on*, May 2013, pp. 1970–1975.

[5] N. Vahrenkamp, D. Muth, P. Kaiser, and T. Asfour, “Ik-map: An enhanced workspace representation to support inverse kinematics solvers,” in *Humanoid Robots (Humanoids), 2015 IEEE-RAS 15th International Conference on*, Nov 2015, pp. 785–790.

[6] F. Zacharias, C. Borst, M. Beetz, and G. Hirzinger, “Positioning mobile manipulators to perform constrained linear trajectories,” in *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Sept 2008, pp. 2578–2584.

[7] F. Zacharias, C. Borst, and G. Hirzinger, “Capturing robot workspace structure: representing robot capabilities,” in *2007 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Oct 2007, pp. 3229–3236.

[8] F. Zacharias, D. Leidner, F. Schmidt, C. Borst, and G. Hirzinger, “Exploiting structure in two-armed manipulation tasks for humanoid robots,” in *Intelligent Robots and Systems (IROS), 2010 IEEE/RSJ International Conference on*, Oct 2010, pp. 5446–5452.

Authorized licensed use limited to: Universidad de Costa Rica (UCR). Downloaded on June 18,2021 at 16:16:41 UTC from IEEE Xplore. Restrictions apply.